



Cal Poly
Pomona

College of Engineering

ECE 4301.01

Fall 2025

Dr. Mohammed E. Aly

Quiz 2 - Crypto on Chip SHA-1 & SHA-256 on Raspberry Pi 5 (Rust vs. “Engine”)

By Quinn Bell, Melvin Contreras, Aaron Tran

Setup & Method

Before building the project, Rust must be installed on the Raspberry Pi 5 using the official installation script. This can be done by running the curl command to download and execute the Rust installer, followed by sourcing the cargo environment. Once Rust is installed, two separate binaries need to be built to enable performance comparison.

The software-only version is built using the RUSTFLAGS environment variable set to "-C target-cpu=generic" which forces the compiler to use generic CPU features without hardware acceleration. This binary is then moved and renamed to "quiz2-soft". The hardware-accelerated version is built with RUSTFLAGS set to "-C target-cpu=native" which enables the compiler to utilize all available CPU features including AArch64 SHA instructions. This binary is renamed to "quiz2-accel". Both builds use the --release flag to ensure optimizations are enabled for accurate performance measurements.

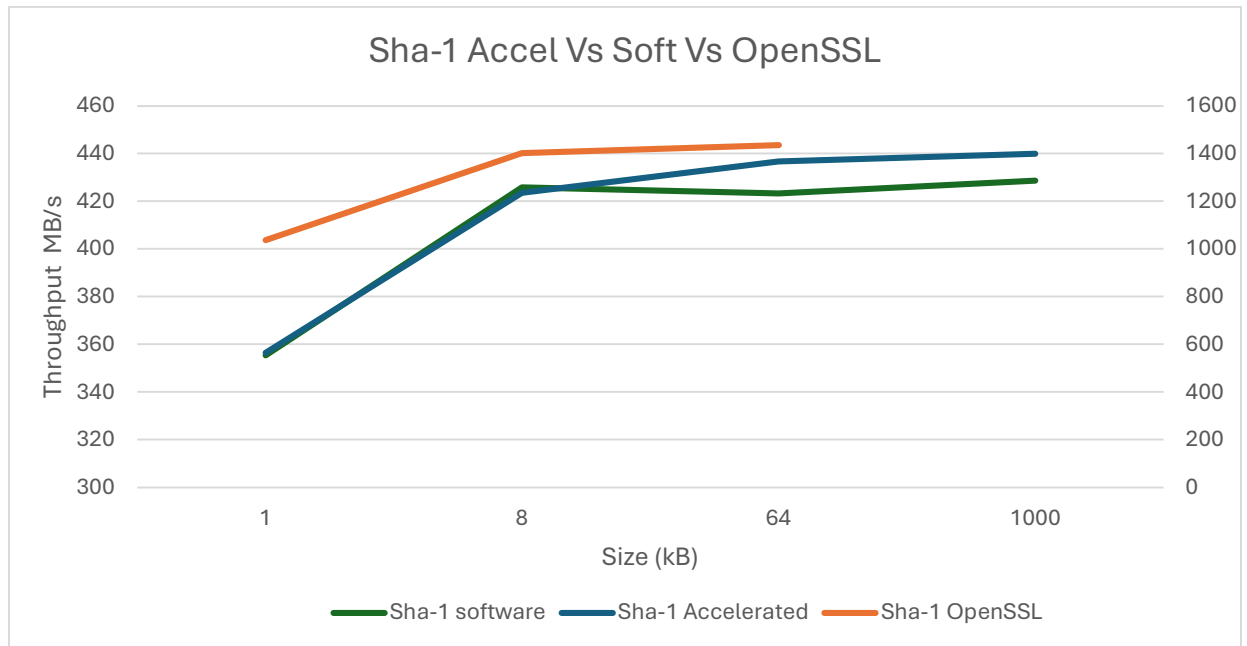
To ensure consistent and reliable benchmark results, the tool should be executed using the taskset command to pin the process to a single CPU core (typically core 3). This eliminates variability caused by CPU migrations and frequency scaling. The benchmark command runs multiple trials (default 5, but 10 is recommended for better statistical validity) on each buffer size and reports the throughput in MB/s along with sample hash digests.

For the camera demonstration workflow, images are first captured using the rplicam-still command (or libcamera-still as an alternative). Multiple images should be captured to provide sufficient test data. Each image is then hashed using both the software-only and hardware-accelerated versions of the tool, with results redirected to separate output files for comparison. The timing data shows the performance difference between the two implementations.

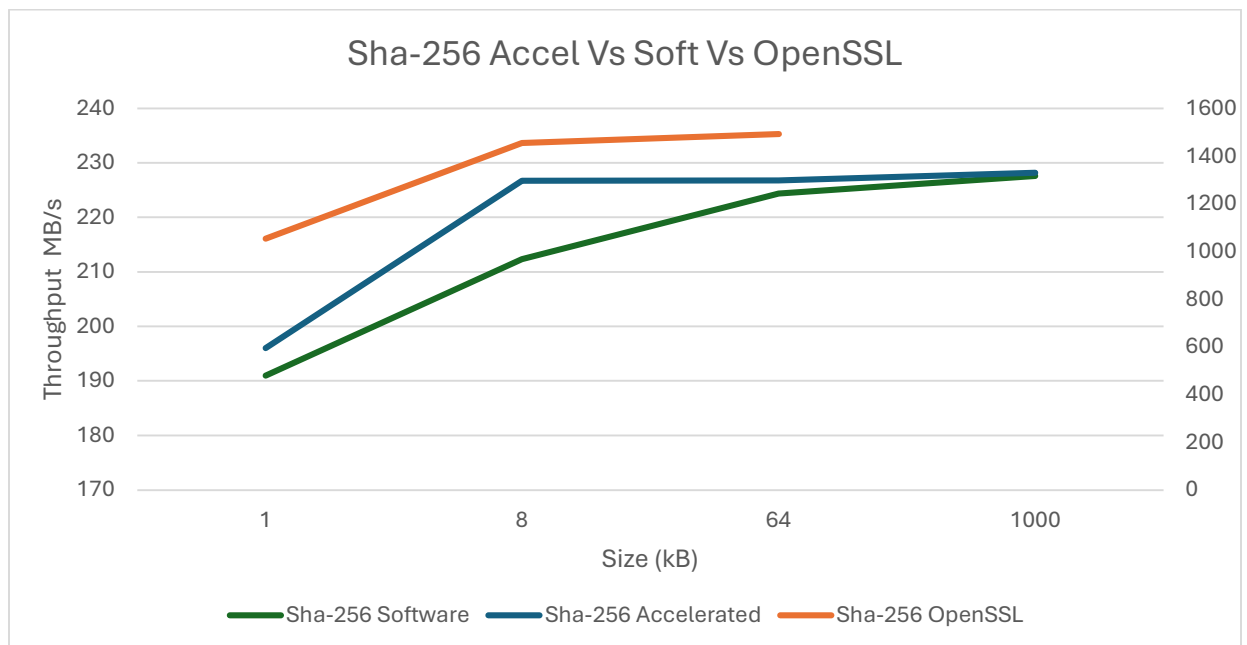
To establish a performance baseline and validate the Rust implementation, OpenSSL's built-in speed test is used. The "openssl speed -evp sha1" and "openssl speed -evp sha256" commands test the same hashing algorithms across various block sizes and report throughput in thousands of bytes per second. These results provide a reference point to compare against the custom Rust implementation and verify that the hardware acceleration is being properly utilized.

Results and Graphs

Sha 1 Results



Sha 256 Results



Analysis

For the data obtained above, 10 runs were ran for various block sizes of 1, 8, 64, and 1000 kilobytes. (Individual runs and data can be found on the excel in the GitHub). In green we have the pure software runs and in blue we have the hardware accelerated runs using Hardware acceleration. Finally, we also have the OpenSSL engine speeds to compare against on the graphs as well in orange. OpenSSL values use the Second Y-axis on the right. Both graphs follow this reference with the top graph being for Sha-1 and bottom graph being for Sha-256.

Firstly, looking at the results from our Sha-1 results we see some interesting differences between hardware acceleration and pure software. For the block sizes 1 and 8 kilobytes they seem to have near identical throughputs. At 64 kilobytes there starts to be noticeably larger difference in throughput. The final block size of 1000 kB has the largest noticeable difference in throughput. This may be due to how as we get larger in our block sizes more and more data is being processed at a time allowing for more efficient data processing. If we use larger block sizes, we could potentially continue to see larger and larger differences in throughputs. Comparing it with the OpenSSL benchmark we can see that the inbuilt engine is much faster than both the hardware and software acceleration but follows the same data trend as both. One interesting thing to note for OpenSSL we can see that as the block sizes get larger the improvements in throughput decreases, suggesting a diminishing return after a certain point.

Looking at the results from our Sha-256 results we see a different trend when comparing hardware acceleration to pure software. There is a more noticeable difference in the smaller block sizes of 1 and 8 kilobytes compared to the larger block sizes. At around 64 kilobytes the throughput for both pure software and hardware acceleration seems to be relatively the same. It seems at 1000 kilobytes the 2 throughputs converge to the same point. It can then be assumed that larger and larger block sizes would provide little to no advantage in providing additional throughput to the data. In comparison to OpenSSL, once again the engine has significantly faster speeds than our hardware acceleration and pure software. However, it follows the data trend of both hardware acceleration and pure software once again. OpenSSL for Sha 256 also seems to have diminishing returns after approaching larger and larger block sizes.

Overall, the difference between Hardware and software for both Sha 1 and 256 share differences in throughput depending on block sizes. With Sha-1 the difference is more noticeable in larger block sizes compared to Sha-256's differences being more noticeable in smaller block sizes. Overall, OpenSSL speeds for both are significantly faster and have a diminishing return as block sizes increase.